

Contents

Week 5: Advanced Machine Learning & External Validation	2
AI/ML in Biomarker Discovery — miRNA in Alzheimer’s Disease	2
Learning Objectives	2
Conceptual Overview: Why Weeks 1–4 Are Not Enough	2
MODULE 5.1 — The Overfitting Problem	2
5.1.1 The Bias-Variance Tradeoff	2
5.1.2 The Curse of Dimensionality	3
5.1.3 Why a 99% Training Accuracy Is Meaningless	4
5.1.4 Data Leakage: What It Is and Its Five Most Common Forms	4
5.1.5 Checklist to Prevent Data Leakage	5
MODULE 5.2 — Cross-Validation Strategies	5
5.2.1 k-Fold Cross-Validation	5
5.2.2 Stratified k-Fold: Preserving Class Balance Across Folds	7
5.2.3 Leave-One-Out Cross-Validation (LOOCV)	8
5.2.4 Nested Cross-Validation for Unbiased Performance Estimation	9
5.2.5 Visualizing Nested CV vs Flat CV	11
MODULE 5.3 — Two R Classifiers: Random Forest and LASSO	11
5.3.1 Random Forest: Bagging with Random Feature Subsets	11
5.3.2 LASSO Logistic Regression: Regularization for Sparse Biomarker Panels	13
5.3.3 Choosing Between Random Forest and LASSO	14
MODULE 5.4 — SHAP Values for Model Interpretation	15
5.4.1 Why Feature Importance Alone Is Not Enough	15
5.4.2 What Is a SHAP Value?	15
5.4.3 Computing SHAP Values for Random Forest Using fastshap	15
5.4.4 The Beeswarm Plot: Global Model Interpretation	16
5.4.5 SHAP Dependence Plot: How One miRNA’s Effect Varies	18
5.4.6 SHAP vs Gini Feature Importance: Which Should You Report?	18
MODULE 5.5 — Three-Class Classification: AD vs MCI vs Control	19
5.5.1 Why MCI Makes Classification Harder	19
5.5.2 Multiclass Classification Approaches	19
5.5.3 Confusion Matrix for Three Classes	20
MODULE 5.6 — External Validation: Why It Matters	22
5.6.1 The Replication Crisis in Biomarker ML	22
5.6.2 Training AUC vs Validation AUC: Interpreting the Gap	22
5.6.3 The GSE46579 Validation Dataset	23
MODULE 5.7 — Harmonizing Datasets for Cross-Platform Validation	23
5.7.1 The miRNA Name Problem	23
5.7.2 miRBaseConverter Name Harmonization in R	24
5.7.3 Per-Dataset Z-Score Standardization	24
5.7.4 External Validation: Train on GSE120584, Apply to GSE46579	25
5.7.5 ComBat for Batch Correction Between Studies	26
MODULE 5.8 — Reporting ML Results: What Makes a Good Paper	27
5.8.1 TRIPOD: Transparent Reporting of a Multivariable Prediction Model	27
5.8.2 What Metrics to Report	27
5.8.3 Confidence Intervals on AUC and DeLong’s Test	28
5.8.4 Calibration Plots	29
5.8.5 Decision Curve Analysis	30
5.8.6 The STARD Checklist	30
WEEK 5 LAB SESSION	31
Lab 5: Nested Cross-Validation, SHAP, and External Validation in R (180 min)	31
WEEK 5 ASSIGNMENTS	32
Reading Assignment	32
Reflection Questions	32
Practical Exercise	32

WEEK 5 GLOSSARY	32
KEY REFERENCES (Week 5)	34
Next Week Preview (Week 6)	35

Week 5: Advanced Machine Learning & External Validation

AI/ML in Biomarker Discovery — miRNA in Alzheimer’s Disease

Learning Objectives

By the end of Week 5, you will be able to: 1. Explain the bias-variance tradeoff and identify overfitting in ML model results, including the five most common forms of data leakage in bioinformatics 2. Design and implement nested cross-validation to obtain unbiased estimates of model performance and hyperparameter selection 3. Build regularized logistic regression (LASSO) using glmnet and tune the regularization parameter lambda using cross-validation 4. Compute and interpret SHAP values to explain model predictions to a biological audience, connecting computational importance scores to known AD miRNA biology 5. Extend binary classifiers to the three-class AD/MCI/Control problem and critically evaluate performance metrics appropriate for multiclass imbalanced data 6. Conduct a formal external validation analysis using GSE46579 as an independent cohort, applying cross-platform harmonization via miRBaseConverter 7. Report ML-based biomarker results according to TRIPOD and STARD guidelines, including confidence intervals on AUC and DeLong’s test for AUC comparison

Conceptual Overview: Why Weeks 1-4 Are Not Enough

In Weeks 3 and 4, you built your first machine learning classifiers — logistic regression, random forest — and saw training and test set AUC values. Perhaps your random forest achieved AUC = 0.91 on the held-out test set. That is an exciting number. Before you write a paper and send a press release, however, three questions should give you pause:

Question 1: Was your model performance estimate actually unbiased? In high-dimensional biological data — where a typical cohort has perhaps 800 miRNA features and only ~150 patients — many sources of optimistic bias can inflate apparent performance even on a “test set.”

Question 2: Can you explain *which* miRNAs drove the predictions, and do those miRNAs make biological sense? A black-box model that cannot be interpreted is unlikely to generate clinical hypotheses or survive peer review.

Question 3: Does your model work on a completely different cohort, sequenced on a different platform, in a different country? A model that cannot replicate externally is a statistical artifact, not a biomarker.

Week 5 addresses all three questions. We introduce the rigorous statistical machinery that separates publishable biomarker studies from the many that cannot be reproduced. The methods in this week are demanding — but they are exactly what separates good science from overfit models.

MODULE 5.1 — The Overfitting Problem

5.1.1 The Bias-Variance Tradeoff

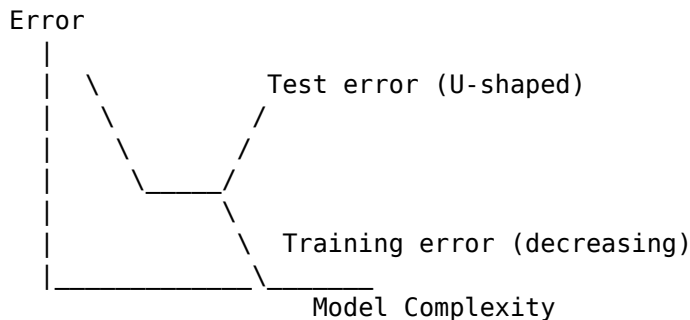
Every supervised ML model makes a tradeoff between two types of error.

Bias is the error that comes from a model being too simple to capture the true relationship in the data. A logistic regression with no interaction terms applied to data with complex non-linear biology has high bias. The model cannot represent the true signal, so it performs poorly on both training and test data. This is called *underfitting*.

Variance is the error that comes from a model being too sensitive to the specific samples in the training data. A decision tree with no depth limit has low bias (it can fit any pattern) but very high variance: change 5 samples and you get a completely different tree. This is called *overfitting*.

The ideal model has moderate, balanced bias and variance. In practice, as model complexity increases: - Training error decreases monotonically (the model “memorizes” the data) - Test error first decreases (the model captures real biology), then increases (it captures noise)

The minimum of the test error curve is where you want to be. Cross-validation, regularization, and ensemble methods are all techniques for finding and staying near that minimum.



In miRNA biomarker discovery: A typical published cohort has roughly 800 detected miRNAs (features) and ~150 patients (samples) — a feature-to-sample ratio of about 5:1, a regime where overfitting is the primary threat. Every model that can freely adjust 800 parameters with only 148 examples will overfit if not carefully constrained. The examples in this module use such a 148-patient cohort. Our own datasets sit on either side: GSE120584 is unusually large (925 miRNAs, 1,296 AD + Control samples), while our validation cohort GSE46579 (273 miRNAs, 65 samples) is in the classic $p > n$ regime. Even with GSE120584’s size, a weak and diffuse signal means that leakage and optimistic tuning still matter.

5.1.2 The Curse of Dimensionality

The “curse of dimensionality” refers to a family of problems that emerge as the number of features grows relative to the number of samples.

Sparsity: In high-dimensional space, all samples become approximately equidistant from each other. Nearest-neighbor classifiers and distance-based clustering stop working well because the concept of “close” loses meaning.

Exponential sample requirements: To densely sample a 1-dimensional space with N points requires N samples. To densely sample a 2-dimensional space requires N^2 . For a d -dimensional space: N^d . With 800 miRNA features, you would theoretically need an astronomical number of samples to densely sample the feature space — but you have 148 patients.

Practical consequences for miRNA ML: - Many machine learning models will appear to perform extremely well on training data simply by finding coincidental correlations among 800 features that happen to separate the 148 training samples - These correlations will not replicate in new patients - The solution is: feature selection/reduction before modeling, strong regularization, and rigorous cross-validation

Biological intuition: If you randomly pick 100 patients and flip a coin to assign them “AD” or “Control” (completely random labels), a random forest with 800 features will still achieve approximately 60–70% accuracy on the training set by memorizing accidental

patterns. This is not biology. This is overfitting. This is why you always evaluate on held-out data.

5.1.3 Why a 99% Training Accuracy Is Meaningless

Consider this scenario: you train a random forest on 148 miRNA expression samples (AD vs Control). You evaluate performance on the same 148 samples the model was trained on. You report “training accuracy = 99%.”

A decision tree with no depth limit will achieve exactly 100% training accuracy on any dataset, every time, by definition. It simply memorizes every sample. This tells you nothing about whether the model learned biology.

The only number that matters is performance on samples the model has never seen.

In practice, because we have small N , we cannot afford to permanently hold out a large test set. Instead, we use **cross-validation** — systematically rotating which samples are held out — to get an unbiased estimate of generalization performance. Module 5.2 covers this in detail.

5.1.4 Data Leakage: What It Is and Its Five Most Common Forms

Data leakage occurs when information from the test set “leaks” into the training process in ways that inflate apparent performance but are not available in real clinical deployment. Leakage is the single most common reason published biomarker studies fail to replicate.

Definition: Any flow of information from the evaluation set (the data used to estimate performance) into the model training process constitutes leakage. Even subtle leakage can inflate AUC by 0.05–0.30, making a mediocre model appear excellent.

The five most common forms of data leakage in bioinformatics:

Form 1: Feature selection before data splitting The most common mistake. You compute differential expression across all 148 patients to select the top 50 miRNAs, then use those 50 miRNAs in a cross-validation loop. Problem: the feature selection step used all 148 samples, including the eventual test fold. The selected features were chosen because they separate *your exact samples*, not because they generalize.

Example: In a 5-fold CV, if you select features using all 148 samples and then cross-validate, your “test fold” samples influenced which features were selected. The features are therefore optimized for those test samples. Your reported AUC can be 0.10–0.25 higher than the true generalizable value.

Correct approach: Feature selection must happen **INSIDE** the cross-validation loop, applied only to the training fold.

Form 2: Normalization using global statistics You compute mean and standard deviation across all 148 samples, then z-score normalize, then split into folds. Problem: the test fold samples contributed to the normalization parameters. In deployment, you would not have access to future patients when normalizing training data.

Correct approach: Fit normalization parameters (mean, SD, scaler) on the training fold only. Apply those parameters to the test fold without refitting.

Form 3: Duplicate/correlated samples across train and test If a study contains technical replicates (the same patient’s RNA run twice), or if two studies share patient samples (which happens more often than you might think), splitting randomly places replicates in both train and test. The model then “sees” essentially the same sample in training and test.

Correct approach: Split by patient, not by sample. Verify no shared patient IDs between your training and external validation datasets.

Form 4: Target-informed imputation If you have missing values in your expression matrix and impute them using global statistics computed across all samples including the test fold, you have leaked label information into the imputed values.

Correct approach: Impute missing values using only training fold statistics. Apply training fold imputation parameters to the test fold.

Form 5: Post-hoc threshold selection You train your model, generate predicted probabilities on the test set, then choose the classification threshold (e.g., 0.45 instead of the default 0.50) that maximizes accuracy on that same test set. The threshold was selected using the test set. If you then report accuracy at that threshold, you have overfit the threshold to the test set.

Correct approach: Select the operating threshold on a validation fold or through the training CV loop. Report test set performance at a fixed or clinically pre-specified threshold.

5.1.5 Checklist to Prevent Data Leakage

Before finalizing any analysis, verify each of the following:

Pipeline construction: - [] Feature selection (DESeq2, variance filter, or any other) occurs INSIDE the CV loop, applied only to training data - [] Normalization parameters (mean, SD, scaler) are fit ONLY on training data and applied to test data - [] Hyperparameter tuning uses ONLY training data (nested CV or a dedicated validation fold) - [] The test fold is touched EXACTLY ONCE — at the end, to report performance

Sample integrity: - [] Training and external validation patients are confirmed non-overlapping - [] No technical replicates of the same patient appear in both train and test - [] Paired samples (e.g., baseline and follow-up from the same patient) are kept together in the same fold

Reporting: - [] All reported metrics are from held-out data, not training data - [] If thresholds were tuned, performance is reported at the TRAINING-fold-selected threshold, not the test-fold-optimal threshold - [] The number of times the test set was evaluated is tracked (ideally: once)

Biologist's perspective: Data leakage in genomics is analogous to a clinical trial where the control group reads the case report forms before their evaluation, so they know what answers the study is looking for. The “result” of that trial is meaningless. Leakage produces a meaningless model — one that appears to work on paper but will fail the moment you apply it to a new patient.

MODULE 5.2 — Cross-Validation Strategies

5.2.1 k-Fold Cross-Validation

k-Fold cross-validation is the standard method for estimating model performance when the dataset is too small to afford a permanent held-out test set.

How it works: 1. Randomly divide the N samples into k equal-sized groups (“folds”) 2. For each fold i from 1 to k: a. Train the model on all samples NOT in fold i (N - N/k training samples) b. Evaluate on fold i (N/k test samples) c. Record the performance metric (AUC, accuracy, etc.) 3. Average performance across all k folds → estimated generalization performance

k = 5 is the most common choice. **k = 10** is more computationally expensive but produces a slightly lower-variance estimate. For a dataset of 148 samples: - **k = 5:** each fold has ~30 test samples; trains on ~118 - **k = 10:** each fold has ~15 test samples; trains on ~133

Important: In our course, k-fold CV is used for performance estimation. When it is also used for hyperparameter tuning without nesting, it becomes biased (see Section 5.2.4).

```
library(caret)
library(randomForest)
```

```

# Load preprocessed training data (from Week 2/3 pipeline)
expr_train <- readRDS("data/processed/harmonized_expr.rds")
meta_train <- readRDS("data/processed/metadata_harmonized.rds")

# Binary classification: AD vs Control (drop MCI – covered in Module 5.5)
mask <- meta_train$group %in% c("Alzheimer's Disease", "Control")
X <- t(expr_train[, meta_train$geo_accession[mask]]) # samples x miRNAs
y <- factor(
  ifelse(meta_train$group[mask] == "Alzheimer's Disease", "AD", "Control"),
  levels = c("Control", "AD")
)

cat("Training set:", nrow(X), "samples,", ncol(X), "miRNA features\n")
cat("Class distribution:\n")
print(table(y))

# 5-fold cross-validation using caret
set.seed(42)
ctrl_basic <- trainControl(
  method = "cv",
  number = 5,
  classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

# NOTE: This is a "flat" CV with fixed hyperparameters.
#       Acceptable for evaluation when hyperparameters are pre-specified.
#       Biased if hyperparameters were tuned using the same data (see Section 5.2.4).
model_rf_flat <- train(
  x = X,
  y = y,
  method = "rf",
  metric = "ROC",
  trControl = ctrl_basic,
  tuneGrid = data.frame(mtry = floor(sqrt(ncol(X)))), # fixed mtry
  ntree = 100
)

cv_auc <- max(model_rf_flat$results$ROC)
cat(sprintf("\n5-Fold CV AUC: %.4f\n", cv_auc))

# Per-fold AUC from saved predictions
preds <- model_rf_flat$pred
fold_aucs <- sapply(unique(preds$Resample), function(fold) {
  d <- preds[preds$Resample == fold, ]
  as.numeric(pROC::auc(pROC::roc(d$obs, d$AD, direction = "<", quiet = TRUE)))
})
cat("Per-fold AUC:", round(fold_aucs, 4), "\n")
cat("Mean AUC:      ", round(mean(fold_aucs), 4), "\n")
cat("SD AUC:        ", round(sd(fold_aucs), 4), "\n")
cat("95% CI (approx):", round(mean(fold_aucs) - 1.96 * sd(fold_aucs), 4),
    "\n", round(mean(fold_aucs) + 1.96 * sd(fold_aucs), 4), "\n")

```

5.2.2 Stratified k-Fold: Preserving Class Balance Across Folds

The problem with standard k-fold on imbalanced data: If 40% of your samples are AD, 40% MCI, and 20% Control, random fold assignment can produce a fold where one class has very few or zero representatives. Training on a fold that has no Control samples, or evaluating on a fold with only 2 Control samples, produces highly variable and unreliable performance estimates.

Stratified k-fold ensures that each fold has the same class proportions as the full dataset. If the dataset is 40% AD, every fold will also be approximately 40% AD.

```
library(caret)

# caret's trainControl always uses stratified CV for classification by default
# when method = "cv" or "repeatedcv". No extra argument is needed.
# You can verify by inspecting fold class distributions from saved predictions.

set.seed(42)
ctrl_stratified <- trainControl(
  method      = "cv",
  number      = 5,
  classProbs  = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

model_rf_strat <- train(
  x      = X,
  y      = y,
  method = "rf",
  metric  = "ROC",
  trControl = ctrl_stratified,
  tuneGrid = data.frame(mtry = floor(sqrt(ncol(X)))),
  ntree    = 100
)

cat(sprintf("Stratified 5-Fold CV AUC: %.4f\n",
           max(model_rf_strat$results$ROC)))

# Verify stratification: check class balance in each fold
preds_strat <- model_rf_strat$pred
cat("\nClass distribution in each fold (verification):\n")
for (fold in unique(preds_strat$Resample)) {
  d <- preds_strat[preds_strat$Resample == fold, ]
  tbl <- table(d$obs)
  pct <- round(prop.table(tbl) * 100)
  cat(sprintf(" %s: Control=%d (%d%%), AD=%d (%d%%)\n",
             fold, tbl["Control"], pct["Control"], tbl["AD"], pct["AD"]))
}
```

Why this matters biologically: Alzheimer's disease datasets frequently have unequal group sizes because MCI patients are harder to recruit and controls are often over-represented in convenience samples. If your test fold happens to contain all MCI patients and no controls, your AUC for that fold is measuring something completely different from the other folds — the average across folds will be meaningless.

5.2.3 Leave-One-Out Cross-Validation (LOOCV)

LOOCV is the extreme case of k-fold where $k = N$ (number of samples). Each iteration trains on $N-1$ samples and tests on exactly one sample. This is repeated N times, once for each sample.

When to use LOOCV: - Very small datasets ($N < 50$): minimizes the amount of data withheld per iteration - When you need every sample in training to avoid high-bias estimates - Simple, computationally fast models (LOOCV with random forest on large datasets is slow)

Limitations of LOOCV: - High variance: each fold shares $N-1$ training samples with every other fold, making fold results highly correlated - For AUC estimation, this high correlation means the variance of the LOOCV-AUC estimate is typically larger than for $k=10$ CV despite using more data per fold - Not appropriate for hyperparameter tuning (same nesting issue as flat k-fold)

```
library(caret)
library(pROC)

# LOOCV – use for small datasets (N < 50)
# With N = 148, LOOCV is computationally feasible but 5-fold is preferred.

set.seed(42)
ctrl_loocv <- trainControl(
  method      = "LOOCV",
  classProbs  = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

# Use LASSO logistic regression for LOOCV (faster than random forest)
glmnet_grid_loo <- expand.grid(alpha = 1, lambda = 0.01)

model_lasso_loo <- train(
  x      = X,
  y      = y,
  method = "glmnet",
  metric = "ROC",
  trControl = ctrl_loocv,
  tuneGrid = glmnet_grid_loo,
  family   = "binomial"
)

# Aggregate LOOCV predictions into one ROC curve
preds_loo <- model_lasso_loo$pred
loo_roc   <- roc(preds_loo$obs, preds_loo$AD, direction = "<", quiet = TRUE)
loo_auc   <- as.numeric(auc(loo_roc))

cat(sprintf("LOOCV AUC (LASSO Logistic Regression): %.4f\n", loo_auc))
cat("Note: LOOCV AUC uses all predicted probabilities to build one ROC curve.\n")
cat("      This is NOT the same as averaging per-fold AUC (undefined for N=1 test).\n")
```

The right tool for the right sample size:

Sample size (N)	Recommended CV strategy
$N < 50$	LOOCV
$50 \leq N < 100$	10-fold stratified CV
$N \geq 100$	5-fold or 10-fold stratified CV
$N \geq 500$	Fixed train/validation/test split (70/15/15)

5.2.4 Nested Cross-Validation for Unbiased Performance Estimation

This is the most important concept in this week. Misunderstanding this point is responsible for a large fraction of inflated performance estimates in the published biomarker literature.

The problem: Suppose you want to: 1. Select the best hyperparameters for your random forest (e.g., `ntree`, `mtry`) 2. Estimate the AUC of the best model on held-out data

If you use the same CV loop for both purposes — selecting the best hyperparameters AND estimating performance — your performance estimate is **biased upward**. The test folds influenced which hyperparameters were selected. The selected hyperparameters are optimal for your specific dataset, including the test folds.

Nested cross-validation solves this by using two loops: - **Outer loop:** Evaluates model performance (produces the reported AUC) - **Inner loop:** Selects hyperparameters using only the outer training fold

The test fold in the outer loop is never touched during hyperparameter selection.

Outer loop ($k=5$):

Outer fold 1: Test = samples 1-30 | Outer train = samples 31-148

Inner loop ($k=5$ within outer train):

Inner fold 1.1: test = 31-53, train = 54-148

Inner fold 1.2: test = 54-76, train = 31-53 + 77-148

...

→ Select best hyperparameters based on inner CV AUC

→ Retrain on outer train (31-148) with best hyperparameters

→ Evaluate on outer test (1-30) → outer AUC for fold 1

Outer fold 2: Test = samples 31-60 | Outer train = samples 1-30 + 61-148

...

Final reported AUC = mean of 5 outer fold AUCs

(Test samples were NEVER used in any hyperparameter selection)

```
library(caret)
```

```
# =====  
# Nested cross-validation implementation in caret  
# =====  
# IMPORTANT: train() on its own is NOT nested CV.  
# caret evaluates every candidate hyperparameter on the SAME resamples  
# (here 5-fold x 3 repeats) and reports the resampled AUC of the best  
# one – tuning and performance estimation share the folds (flat CV).  
# The code below is what Week5_Validation.R does; a truly nested  
# version wraps it in an outer loop (see the snippet after this block).
```

```
set.seed(42)
```

```
# OUTER loop: 5-fold x 3 repeats for stable AUC estimate
```

```
ctrl_nested <- trainControl(  
  method      = "repeatedcv",  
  number      = 5,  
  repeats     = 3,  
  classProbs  = TRUE,  
  summaryFunction = twoClassSummary,  
  savePredictions = "final"  
)
```

```

# Hyperparameter grid for the inner loop search (RF: tuning mtry)
rf_grid <- expand.grid(
  mtry = c(floor(sqrt(ncol(X))),
            floor(ncol(X) / 3),
            floor(ncol(X) / 5))
)

# caret evaluates each mtry value using inner CV (default: bootstrapping
# within the outer training fold). The outer loop estimates AUC.
set.seed(42)
model_rf_nested <- train(
  x      = X,
  y      = y,
  method = "rf",
  metric  = "ROC",
  trControl = ctrl_nested,
  tuneGrid = rf_grid,
  ntree    = 300
)

cat("Nested CV Results (5-fold × 3-repeat outer loop):\n")
print(model_rf_nested$results[, c("mtry", "ROC", "Sens", "Spec")])
cat("\nBest mtry:", model_rf_nested$bestTune$mtry, "\n")
cat("Nested CV AUC (mean over folds):",
    round(max(model_rf_nested$results$ROC, na.rm = TRUE), 4), "\n")

# Per-fold outer AUC
preds_nested <- model_rf_nested$pred %>%
  dplyr::filter(mtry == model_rf_nested$bestTune$mtry)

fold_aucs_nested <- sapply(unique(preds_nested$Resample), function(fold) {
  d <- preds_nested[preds_nested$Resample == fold, ]
  as.numeric(pROC::auc(pROC::roc(d$obs, d$AD, direction = "<", quiet = TRUE)))
})

cat("\nPer-fold outer AUC values:\n")
cat(" ", round(fold_aucs_nested, 4), "\n")
cat(" Mean: ", round(mean(fold_aucs_nested), 4), "\n")
cat(" SD:   ", round(sd(fold_aucs_nested), 4), "\n")
cat(" 95% CI:", round(mean(fold_aucs_nested) - 1.96 * sd(fold_aucs_nested), 4),
    "-", round(mean(fold_aucs_nested) + 1.96 * sd(fold_aucs_nested), 4), "\n")

# Compare to flat CV (fixed, non-tuned hyperparameters)
flat_auc <- max(model_rf_flat$results$ROC, na.rm = TRUE)
cat(sprintf("\nNested CV AUC (tuned):      %.4f\n", max(model_rf_nested$results$ROC)))
cat(sprintf("Flat CV AUC (fixed mtry):    %.4f\n", flat_auc))
cat("(If nested CV AUC is lower, hyperparameter tuning was overfitting the folds.)\n")

```

Making it truly nested. Put `train()` (the inner, tuning loop) inside an outer loop that holds out samples it never sees:

```

set.seed(42)
outer_folds <- createFolds(y, k = 5) # outer loop: 5 held-out folds
outer_auc <- sapply(outer_folds, function(test_idx) {
  fit <- train(x = X[-test_idx, ], y = y[-test_idx, ], # inner loop tunes mtry on
               method = "rf", metric = "ROC",          # the outer TRAINING data only
               trControl = ctrl_nested, tuneGrid = rf_grid, ntree = 300)

```

```

p <- predict(fit, X[test_idx, ], type = "prob")[, "AD"]
as.numeric(pROC::auc(y[test_idx], p, quiet = TRUE))
})
mean(outer_auc) # nested CV AUC

```

How big is the difference for our data? With ~1,300 training samples and only three mtry values, the optimism of caret's flat estimate is small. In GSE120584 the flat CV AUC is 0.74 (Random Forest) and 0.77 (LASSO). The difference matters much more for small cohorts and large tuning grids.

5.2.5 Visualizing Nested CV vs Flat CV

Understanding the architectural difference between these two approaches is essential for designing valid experiments.

FLAT CV (biased when hyperparameters are tuned):

ALL 148 samples

Grid search finds best hyperparameters
using all 148 samples ← LEAKAGE

Same 148 samples used to evaluate performance
using k-fold ← Biased: test folds saw HP tuning

NESTED CV (unbiased):

OUTER FOLD 1 TEST: samples 1–30 (NEVER touched in HP tuning – completely held out)

OUTER FOLD 1 TRAIN: samples 31–148

INNER CV on samples 31–148 only:

Inner fold 1: train 66–148, test 31–65

Inner fold 2: train 31–65+97–148, test 66–96

...

→ Best hyperparameters selected here

Retrain with best HPs on all 31–148

Evaluate on 1–30 → unbiased AUC for fold 1

Repeat for all 5 outer folds → 5 unbiased AUC estimates

Key insight for biologists: The difference between nested and flat CV typically amounts to 0.02–0.08 AUC units in miRNA studies. In a field where AUC 0.75 vs 0.83 is the difference between a mediocre and a strong biomarker, this gap matters enormously. Use nested CV for all published results.

MODULE 5.3 — Two R Classifiers: Random Forest and LASSO

5.3.1 Random Forest: Bagging with Random Feature Subsets

Random Forest is an **ensemble** method that builds many decision trees independently and averages their predictions. You were introduced to it in Week 4; here we focus on the key tuning parameters and their biological interpretation.

How Random Forest works: 1. Draw a bootstrap sample (N samples with replacement) from the training data 2. Grow a decision tree on this bootstrap sample. At each split, consider only a random subset of `mtry` features (not all `p` features). Choose the best split among this random subset. 3. Repeat steps 1-2 to grow `ntree` trees (typically 300-500) 4. Final prediction = majority vote (classification) or mean (regression) across all trees

Key hyperparameters:

Parameter	Typical range	What it controls
<code>ntree</code>	300-1000	Number of trees. More is almost always better; diminishing returns after ~300
<code>mtry</code>	$\sqrt{p}$ to $p/3$	Features sampled at each split. Smaller = more diverse trees; larger = each tree uses more features
<code>min.node.size</code>	1-10	Minimum samples required in a leaf. Larger = simpler trees, less overfitting

Biological rationale for `mtry`: If only 10 of your 800 miRNAs are true AD biomarkers, setting `mtry =  $\sqrt{800} \approx 28$` means each split randomly samples 28 features. On average, $28 \times (10/800) \approx 0.35$ of those are informative. Many splits will have no informative feature in their random sample — but this is actually beneficial. It forces trees to be diverse, and the uninformative miRNAs average out across the ensemble.

```
library(randomForest)
library(pROC)

set.seed(42)

# Fit Random Forest on full training data for illustration
# (For performance estimation, use cross-validation as in Section 5.2)
rf_model <- randomForest(
  x      = X,
  y      = y,
  ntree  = 500,
  mtry   = floor(sqrt(ncol(X))), # recommended default for classification
  importance = TRUE,             # compute variable importance
  keep.forest = TRUE
)

print(rf_model)

# Predicted probabilities on training data (ONLY for illustration – use CV for eval)
prob_train <- predict(rf_model, type = "prob")[, "AD"]
roc_train <- roc(y, prob_train, direction = "<", quiet = TRUE)
cat(sprintf("Training AUC (optimistic, not CV): %.4f\n",
            as.numeric(auc(roc_train))))

# Variable importance: MeanDecreaseGini (impurity-based)
# MeanDecreaseAccuracy (permutation-based) is more reliable but slower
imp_df <- as.data.frame(importance(rf_model)) %>%
  tibble::rownames_to_column("miRNA") %>%
  arrange(desc(MeanDecreaseGini))
```

```
cat("\nTop 10 miRNAs by Gini importance:\n")
print(head(imp_df[, c("miRNA", "MeanDecreaseGini")], 10))
```

5.3.2 LASSO Logistic Regression: Regularization for Sparse Biomarker Panels

Logistic regression models the log-odds of class membership as a linear combination of features:

$$\log(P(\text{AD}) / P(\text{Control})) = \beta_0 + \beta_1 \cdot \text{miRNA}_1 + \beta_2 \cdot \text{miRNA}_2 + \dots + \beta_p \cdot \text{miRNA}_p$$

Standard logistic regression on 800 features with 148 samples will overfit severely because there are more parameters than samples. **Regularization** adds a penalty term to prevent large coefficients.

LASSO (L1 regularization): The penalty is proportional to the sum of absolute coefficient values:

$$\text{Penalized loss} = \text{Log-likelihood} - \lambda \times \sum |\beta_j|$$

The key property of L1 regularization is **sparsity**: as λ increases, more and more coefficients are driven exactly to zero. A LASSO-regularized logistic regression with λ tuned by cross-validation automatically selects a small panel of miRNAs — coefficients for uninformative miRNAs collapse to zero.

Biological significance: LASSO implements automated feature selection as part of the regularization. If 750 of your 800 miRNAs have zero LASSO coefficient, the model is telling you that those miRNAs contribute no independent information to the AD prediction beyond what is already captured by the other 50 miRNAs. This produces a parsimonious, interpretable panel — more suitable for a clinical assay than a model that depends on all 800 features.

```
library(glmnet)
library(pROC)

set.seed(42)

# Encode labels as 0/1 (glmnet requires numeric y for binomial family)
y_numeric <- as.integer(y == "AD") # 1 = AD, 0 = Control

# cv.glmnet: finds optimal lambda by cross-validation
# alpha = 1: LASSO (pure L1 penalty)
# alpha = 0: ridge (pure L2 penalty)
# alpha between 0 and 1: elastic net
lasso_cv <- cv.glmnet(
  x      = as.matrix(X),
  y      = y_numeric,
  family = "binomial",
  alpha  = 1,           # LASSO
  nfolds = 10,
  type.measure = "auc", # optimize for AUC (instead of deviance)
  standardize = FALSE   # data already z-scored; do not re-standardize
)

cat("LASSO cross-validation results:\n")
cat("  lambda.min (maximizes AUC):", round(lasso_cv$lambda.min, 6), "\n")
cat("  lambda.1se (1-SE rule, sparser):", round(lasso_cv$lambda.1se, 6), "\n")
cat("  CV AUC at lambda.min:", round(max(lasso_cv$cvm), 4), "\n")

# Plot lambda path (AUC vs log(lambda))
plot(lasso_cv, main = "LASSO Cross-Validation: AUC vs log(lambda)")
```

```

# Coefficients at lambda.min – non-zero entries = selected miRNAs
coefs_min <- coef(lasso_cv, s = "lambda.min")
selected <- rownames(coefs_min)[coefs_min[, 1] != 0]
selected <- selected[selected != "(Intercept)"]
cat("\nmiRNAs selected by LASSO (lambda.min):", length(selected), "\n")
cat("First 10 selected:\n")
print(head(selected, 10))

# Coefficients at lambda.1se – more conservative, fewer features
coefs_1se <- coef(lasso_cv, s = "lambda.1se")
n_selected_1se <- sum(coefs_1se[, 1] != 0)
cat("\nmiRNAs selected at lambda.1se:", n_selected_1se, "\n")
cat("(lambda.1se gives a sparser model within 1 SE of the best CV AUC)\n")

# Coefficient path plot (how coefficients change as lambda varies)
plot(lasso_cv$glmnet.fit,
     xvar = "lambda",
     label = FALSE,
     main = "LASSO Coefficient Path\n(each line = one miRNA)")
abline(v = log(lasso_cv$lambda.min), col = "firebrick", lty = 2)

```

Biological sidebar — What does a LASSO coefficient mean for a miRNA?

A LASSO coefficient of +0.45 for hsa-miR-21-5p means: in log-odds units, each 1-unit increase in z-scored hsa-miR-21-5p expression increases the log-odds of AD by 0.45, holding all other miRNAs constant. Positive coefficients indicate that higher expression predicts AD; negative coefficients indicate that lower expression (or that miRNA is enriched in Controls). LASSO coefficients give both feature selection (non-zero = selected) and effect direction (sign) in one step.

5.3.3 Choosing Between Random Forest and LASSO

Both methods are appropriate for miRNA biomarker discovery. The choice depends on the scientific goal:

Property	Random Forest	LASSO
Feature interactions captured	Yes (via tree splits)	No (linear model)
Interpretable coefficients	No (black box)	Yes (coefficients with direction)
Automatic feature selection	Partial (via importance)	Hard (exact sparsity)
Performance on small N	Good	Very good (regularization handles small N well)
Extrapolation beyond training data	Poor	Better (linear)
Clinical panel design	Rank by importance	Direct: select non-zero features

Recommended strategy: Train both models using nested CV. Compare their AUC values. If LASSO achieves comparable AUC to Random Forest, prefer LASSO for its interpretability and the explicit sparse panel it provides. If Random Forest is substantially better (> 0.05 AUC), the data likely contains non-linear structure or feature interactions that LASSO cannot capture.

MODULE 5.4 — SHAP Values for Model Interpretation

5.4.1 Why Feature Importance Alone Is Not Enough

The Gini-based variable importance from Random Forest tells you which miRNAs are most frequently used by the model and contribute most to prediction accuracy. However, it does not tell you: - Does high expression of this miRNA predict AD, or low expression? - Is this miRNA's effect consistent across all patients, or does it only matter for a subset? - For patient X specifically, which miRNAs drove the prediction toward AD?

SHAP (SHapley Additive exPlanations) values answer all of these questions. They provide a theoretically grounded, interpretable decomposition of every model prediction into contributions from each feature.

5.4.2 What Is a SHAP Value?

SHAP values are derived from cooperative game theory. In game theory, a Shapley value answers: "If multiple players contribute cooperatively to produce an outcome, what is each player's fair share of that outcome?"

In ML, the "players" are the model features (miRNAs), and the "outcome" is the model's prediction for a single patient. A SHAP value for feature j in patient i represents:

"How much did miRNA j shift the model's prediction for patient i away from the average prediction across all patients?"

Specifically, for a predicted probability output: - SHAP value = 0: this miRNA had no effect on this patient's prediction - SHAP value = +0.12: this miRNA pushed the prediction toward AD (increased $P(\text{AD})$ by ~ 0.12) - SHAP value = -0.08: this miRNA pushed the prediction toward Control (decreased $P(\text{AD})$ by ~ 0.08)

The SHAP values for all features in a patient sum to (prediction - base value), where the base value is the model's average prediction across the training set.

5.4.3 Computing SHAP Values for Random Forest Using fastshap

```
library(fastshap)
library(randomForest)
library(ggplot2)
library(tidyr)
library(dplyr)

set.seed(42)

# Refit RF on full training data (with best mtry from nested CV)
rf_final <- randomForest(
  x      = X,
  y      = y,
  ntree  = 500,
  mtry   = floor(sqrt(ncol(X))),
  importance = TRUE
)

# Define a prediction function that fastshap will call
# It must return a numeric vector of predicted probabilities (one per sample)
pfun_rf <- function(object, newdata) {
  predict(object, newdata = as.matrix(newdata), type = "prob")[, "AD"]
}
```

```

# Compute SHAP values via Monte Carlo sampling
# Larger nsim = more precise SHAP but slower. Run time also grows with the
# number of samples explained: with ~1,300 training samples we explain a
# class-balanced random subset of 200 (the background X is still all samples).
set.seed(42)
shap_idx <- c(sample(which(y == "AD"), 100), sample(which(y == "Control"), 100))
X_shap <- as.matrix(X[shap_idx, ])

cat("Computing SHAP values (nsim=50)...\\n")
set.seed(42)
shap_values <- fastshap::explain(
  object      = rf_final,
  feature_names = colnames(X),
  X           = as.matrix(X),      # background data
  newdata     = X_shap,           # samples to explain
  pred_wrapper = pfun_rf,         # NB: the argument is pred_wrapper, not pred_fun
  nsim        = 50
)

# shap_values: samples x features matrix
# Each entry = SHAP contribution of that miRNA for that sample
cat("SHAP dimensions:", dim(shap_values), "\\n")
cat(" (rows = samples, columns = features/miRNAs)\\n")

# Global feature importance: mean |SHAP|
mean_abs_shap <- colMeans(abs(shap_values))
shap_imp <- data.frame(
  miRNA      = names(mean_abs_shap),
  mean_abs_shap = as.numeric(mean_abs_shap)
) %>% arrange(desc(mean_abs_shap))

cat("\\nTop 20 miRNAs by mean |SHAP|:\\n")
print(head(shap_imp, 20))

# Patient-level SHAP for one sample (the most confidently predicted AD patient)
pred_probs <- predict(rf_final, type = "prob")[, "AD"]
top_ad_idx <- which.max(pred_probs)

cat(sprintf("\\nPatient %d: P(AD) = %.4f, true label = %s\\n",
            top_ad_idx, pred_probs[top_ad_idx], y[top_ad_idx]))

patient_shap <- shap_values[top_ad_idx, ]
sorted_idx <- order(abs(patient_shap), decreasing = TRUE)
cat("Top 5 contributing miRNAs for this patient:\\n")
for (i in 1:5) {
  j <- sorted_idx[i]
  cat(sprintf(" %s: SHAP = %.4f (expression = %.4f)\\n",
              names(patient_shap)[j], patient_shap[j], X[top_ad_idx, j]))
}

```

5.4.4 The Beeswarm Plot: Global Model Interpretation

The beeswarm plot (also called the SHAP summary plot) is the single most informative visualization for understanding a model's overall behavior.

How to read it: - Each dot represents one patient-miRNA combination - The x-axis shows the

SHAP value (positive = pushes toward AD, negative = pushes toward Control) - The y-axis lists features ordered by mean |SHAP value| (most important at top) - The color encodes the actual expression level of that miRNA in that patient (blue = low expression, red = high expression)

```
library(ggplot2)
library(tidyr)
library(dplyr)

# Select top 20 miRNAs by mean |SHAP|
top20 <- head(shap_imp$miRNA, 20)

# Reshape to long format
shap_long <- as.data.frame(shap_values[, top20]) %>%
  mutate(sample_idx = seq_len(nrow(X))) %>%
  pivot_longer(cols = -sample_idx, names_to = "miRNA", values_to = "shap_value")

expr_long <- as.data.frame(X[, top20]) %>%
  mutate(sample_idx = seq_len(nrow(X))) %>%
  pivot_longer(cols = -sample_idx, names_to = "miRNA", values_to = "expression")

plot_data <- left_join(shap_long, expr_long, by = c("sample_idx", "miRNA")) %>%
  mutate(miRNA = factor(miRNA, levels = rev(top20)))

# Add vertical jitter within each miRNA row
set.seed(42)
plot_data <- plot_data %>%
  group_by(miRNA) %>%
  mutate(y_jitter = as.numeric(miRNA) + runif(n(), -0.35, 0.35)) %>%
  ungroup()

p_beeswarm <- ggplot(plot_data,
  aes(x = shap_value, y = y_jitter, colour = expression)) +
  geom_point(size = 0.9, alpha = 0.7) +
  geom_vline(xintercept = 0, linetype = "dashed", colour = "grey40") +
  scale_colour_gradient2(
    low = "#4575B4", # blue = low expression
    mid = "white",
    high = "#D73027", # red = high expression
    midpoint = 0,
    name = "Z-scored\nexpression"
  ) +
  scale_y_continuous(
    breaks = seq_along(levels(plot_data$miRNA)),
    labels = levels(plot_data$miRNA)
  ) +
  labs(
    x = "SHAP value (positive = pushes toward AD prediction)",
    y = NULL,
    title = "SHAP Beeswarm Plot – Random Forest AD vs Control\n(Top 20 miRNAs by mean |SHAP value|)"
  ) +
  theme_bw(base_size = 10) +
  theme(panel.grid.minor = element_blank(),
    legend.position = "right")

print(p_beeswarm)
ggsave("results/Week5/shap_beeswarm.png",
```

```
p_beeswarm, width = 9, height = 7, dpi = 150)
```

5.4.5 SHAP Dependence Plot: How One miRNA's Effect Varies

A dependence plot shows how the SHAP value for one miRNA changes as its expression level varies. This reveals non-linear effects and interactions.

```
# SHAP dependence plot for the top feature
top_mirna <- shap_imp$miRNA[1]

dep_data <- data.frame(
  expression = X[, top_mirna],
  shap_value = shap_values[, top_mirna],
  group      = y
)

p_dep <- ggplot(dep_data, aes(x = expression, y = shap_value, colour = group)) +
  geom_point(size = 1.8, alpha = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "grey50") +
  geom_smooth(method = "loess", se = TRUE,
             colour = "black", fill = "grey80", alpha = 0.3, linewidth = 0.8) +
  scale_colour_manual(
    values = c("AD" = "#D73027", "Control" = "#4575B4"),
    name   = "True Group"
  ) +
  labs(
    x      = paste0("Z-scored expression: ", top_mirna),
    y      = paste0("SHAP value for ", top_mirna),
    title  = paste0("SHAP Dependence Plot: ", top_mirna)
  ) +
  theme_bw(base_size = 11)

print(p_dep)
ggsave("results/Week5/shap_dependence_top_feature.png",
       p_dep, width = 7, height = 5, dpi = 150)
```

5.4.6 SHAP vs Gini Feature Importance: Which Should You Report?

Property	Gini importance (RF)	LASSO coefficients	SHAP values
Accounts for feature interactions	No	No	Yes
Direction of effect (up vs down in AD)	No	Yes	Yes
Patient-level explanations	No	No	Yes
Consistent across model types	No	No	Yes (model-agnostic)
Computationally expensive	Fast	Fast	Moderate

Property	Gini importance (RF)	LASSO coefficients	SHAP values
Appropriate for peer-reviewed biomarker papers	Acceptable	Acceptable	Preferred

Recommendation: Report SHAP values in published work. Use Gini importance for internal exploration and sanity checks. Use LASSO coefficients when the sparsity and direction of a linear panel are scientifically important.

Biological sidebar — What does a high SHAP value for miR-29b mean clinically?

Suppose your beeswarm plot shows that hsa-miR-29b-3p has the largest mean |SHAP value|, and the color gradient shows that LOW expression (blue) is associated with POSITIVE SHAP values (pushes toward AD prediction). This tells you:

Low miR-29b-3p expression in serum predicts AD in your model.

This is biologically plausible. miR-29b is a well-studied neuronal miRNA that regulates BACE1 (the beta-secretase enzyme responsible for amyloid precursor protein cleavage — the first step in Abeta production). Multiple studies have reported downregulation of miR-29 family members in blood and brain tissue of AD patients (Hebert et al., 2008 PNAS; Cogswell et al., 2008 J Alzheimers Dis). Low miR-29b → less repression of BACE1 → more Abeta production → AD pathology.

The SHAP value connects a computational importance score to a specific mechanistic hypothesis. This is the kind of interpretation that makes ML-based biomarker studies scientifically meaningful rather than purely predictive.

MODULE 5.5 — Three-Class Classification: AD vs MCI vs Control

5.5.1 Why MCI Makes Classification Harder

In Week 4, we simplified the problem by training binary classifiers (AD vs Control). In clinical reality, MCI (Mild Cognitive Impairment) is a critical class: these are the patients most likely to benefit from early intervention, and they are the population where biomarker-guided treatment decisions would have the greatest impact.

MCI is biologically ambiguous because: - MCI represents a heterogeneous prodromal stage: some MCI patients progress to AD; others remain stable; a few revert to normal cognition - The miRNA profiles of MCI patients partially overlap with both AD and Control - “Amnesic MCI” (most likely to progress to AD) has a different profile from “non-amnesic MCI”

Expected consequence: Your model will have lower performance for MCI samples than for AD vs Control. This is not a failure of the model. It reflects genuine biological uncertainty. A model that reports 90% accuracy for MCI is almost certainly overfitting.

5.5.2 Multiclass Classification Approaches

One-vs-Rest (OvR): Train three binary classifiers: (AD vs not-AD), (MCI vs not-MCI), (Control vs not-Control). For a new patient, apply all three classifiers and assign the class with the highest predicted probability.

Multinomial: Train a single model that directly outputs probabilities for all three classes simultaneously, with the constraint that probabilities sum to 1. Random Forest handles this natively. glmnet uses multinomial logistic regression when `family = "multinomial"`.

For miRNA data, both approaches perform similarly. Multinomial is generally preferred because it uses all class information simultaneously during training.

```
library(caret)

# Include all three groups
mask_3class <- metadata_120584$group %in%
  c("Alzheimer's Disease", "Mild Cognitive Impairment", "Control")

meta_3c <- metadata_120584[mask_3class, ] %>%
  mutate(group_f = factor(
    dplyr::case_when(
      group == "Alzheimer's Disease" ~ "AD",
      group == "Mild Cognitive Impairment" ~ "MCI",
      group == "Control" ~ "Control"
    ),
    levels = c("Control", "MCI", "AD")
  ))

X_3c <- t(expr_120584_z[, meta_3c$geo_accession])
y_3c <- meta_3c$group_f

cat("Three-class label distribution:\n")
print(table(y_3c))

set.seed(42)
ctrl_mc <- trainControl(
  method = "repeatedcv",
  number = 5,
  repeats = 3,
  classProbs = TRUE,
  summaryFunction = multiClassSummary, # multiclass metrics
  savePredictions = "final"
)

model_rf_3c <- train(
  x = X_3c,
  y = y_3c,
  method = "rf",
  metric = "AUC",
  trControl = ctrl_mc,
  tuneGrid = data.frame(mtry = floor(sqrt(ncol(X_3c)))),
  ntree = 300,
  importance = TRUE
)

cat("\nThree-class Random Forest – CV results:\n")
cat("  Macro AUC:      ", round(model_rf_3c$results$AUC, 4), "\n")
cat("  Balanced Acc:    ", round(model_rf_3c$results$Mean_Balanced_Accuracy, 4), "\n")
```

5.5.3 Confusion Matrix for Three Classes

```
library(caret)

# Get predictions at best hyperparameter
preds_3c <- model_rf_3c$pred %>%
```

```

dplyr::filter(mtry == model_rf_3c$bestTune$mtry) %>%
dplyr::group_by(rowIndex) %>%
dplyr::slice(1) %>%      # first repeat (for illustration)
dplyr::ungroup()

cm_3c <- confusionMatrix(preds_3c$pred, preds_3c$obs, mode = "everything")
cat("\nThree-class confusion matrix (one repeat):\n")
print(cm_3c$table)

cat("\nPer-class statistics:\n")
print(round(cm_3c$byClass[, c("Sensitivity", "Specificity", "F1",
                             "Balanced Accuracy")], 3))

# Heatmap of confusion matrix
cm_df <- as.data.frame(cm_3c$table)
colnames(cm_df) <- c("Predicted", "Actual", "Count")

# Row-normalize to get percentages
cm_df <- cm_df %>%
  dplyr::group_by(Actual) %>%
  dplyr::mutate(Pct = Count / sum(Count) * 100) %>%
  dplyr::ungroup()

p_cm <- ggplot(cm_df, aes(x = Predicted, y = Actual, fill = Pct)) +
  geom_tile(colour = "white", linewidth = 0.8) +
  geom_text(aes(label = sprintf("%.1f%%\n(n=%d)", Pct, Count)),
            size = 4, fontface = "bold") +
  scale_fill_gradient(low = "white", high = "#4575B4",
                     name = "Row %", limits = c(0, 100)) +
  labs(
    x = "Predicted Class",
    y = "True Class",
    title = "Confusion Matrix (Row %) – AD/MCI/Control\n5-fold CV (first repeat)"
  ) +
  theme_bw(base_size = 12)

print(p_cm)
ggsave("results/Week5/confusion_matrix_3class.png",
       p_cm, width = 6, height = 5, dpi = 150)

```

Biological interpretation of MCI misclassification: In a typical confusion matrix for this three-class problem, you will see that MCI samples are misclassified as either AD or Control at much higher rates than AD or Control samples are misclassified as each other. This is not a model deficiency — it accurately reflects biological reality. MCI patients are biologically intermediate: some have early amyloid pathology indistinguishable from AD; others have mild, non-specific decline. A model that correctly classifies MCI 55% of the time but AD and Control 85–90% of the time is performing appropriately given the biology. The MCI classification performance tells you something about the biological distinctiveness of MCI as a molecular state — and in this case, the answer is that miRNA alone captures only part of the MCI signature. Combined biomarkers (miRNA + CSF Aβeta/tau, or miRNA + PET imaging) would likely improve MCI classification substantially.

What GSE120584 shows: with 1,009 AD, 287 Control and only 32 MCI samples, the three-class Random Forest **never predicts MCI**: all 32 MCI samples are called AD (MCI sensitivity 0), Controls are mostly called AD too (sensitivity 0.16), and AD sensitivity is 0.99. Overall accuracy (78%) is barely above the 76% obtained by calling everyone

AD. Class weights, down-sampling (`trainControl(sampling = "down")`), a larger MCI cohort, and balanced metrics (balanced accuracy, macro-F1, per-class recall) are needed.

MODULE 5.6 — External Validation: Why It Matters

5.6.1 The Replication Crisis in Biomarker ML

A sobering statistic: a systematic review by Bzdok & Ioannidis (2017) estimated that more than 85% of published “predictive biomarker” studies based on omics data do not include external validation on an independent cohort. Among studies that do attempt external validation, a significant fraction show AUC drops of 0.10–0.25 relative to the training/internal validation estimate.

Why does this happen? Several compounding factors: - **Overfitting** to the training cohort’s specific demographics, sample processing pipeline, and platform characteristics - **Platform-specific technical artifacts** mistaken for biological signal - **Cohort-specific confounders** (e.g., medication profiles, geographic ancestry, comorbidities) that differ between cohorts - **Small training N** producing high-variance models that fitted noise

External validation is not just “a good idea” — it is the minimum standard of evidence required for clinical translation of a biomarker. The FDA’s Biomarker Qualification Guidance explicitly requires independent validation before a biomarker can be used in drug development.

5.6.2 Training AUC vs Validation AUC: Interpreting the Gap

When you apply your model trained on GSE120584 to the independent cohort GSE46579, you will observe an AUC gap:

Training CV AUC (GSE120584): e.g., 0.88
External Validation AUC (GSE46579): e.g., 0.73
Gap: 0.15

How should you interpret the size of this gap?

Gap size	Interpretation
< 0.05	Excellent generalizability. Model likely learned true biology. Publishable with appropriate caveats.
0.05–0.10	Acceptable. Some platform/cohort effects present but model retains predictive value.
0.10–0.20	Moderate degradation. Likely a combination of overfitting and platform/cohort differences. Report honestly. Investigate what drives the loss.
> 0.20	Severe degradation. Model has likely overfit to training cohort. Feature harmonization and retraining may help.
Validation AUC $\leq$ 0.55	Model has failed external validation. Report as a negative result and investigate root cause.

Biological sidebar — What does a 15% AUC drop in external validation mean biologically?

Suppose your model achieves AUC = 0.88 on GSE120584 (serum microarray) but drops to AUC = 0.73 on GSE46579 (whole-blood small RNA-seq). Several biological and technical explanations are possible:

1. **Sample-type and platform difference:** Serum versus whole blood captures different miRNA compartments, and a fluorescence array and sequencing measure them differently (compressed array fold changes; read counts dominated by a few

abundant miRNAs). Serum is depleted of cellular miRNAs and enriched for exosome-packaged miRNAs. Whole blood includes miRNAs from leukocytes. Some miRNAs differentially expressed in serum-AD may not be differentially expressed (or may not be detected) in whole blood.

2. **Cohort heterogeneity:** GSE46579 was collected in a different country, with different AD diagnostic criteria (perhaps including more mild-stage patients), different comorbidity profiles, and different blood processing protocols.
3. **Overfitting in training:** Your model learned some features that are informative in the training cohort but are technical artifacts (e.g., platform-specific background noise patterns that correlate with disease group due to batch effects that were imperfectly corrected).
4. **True biological difference:** Different AD subtypes may have different miRNA profiles, and the cohorts may have different proportions of subtypes.

The most productive response to a 15% drop is not to dismiss the result but to investigate it: examine which features have the highest SHAP values in the training set, check whether those same miRNAs are detected in the validation platform, and test whether a model retrained only on features present in both platforms narrows the gap.

What actually happens with our data: the gap is far larger than in this example. Training CV AUC is 0.74 (Random Forest) and 0.77 (LASSO), but external AUC on GSE46579 is **0.57** (95% CI 0.44–0.69) and **0.42** — no better than chance. The reason shows up in a simple check (Week 5 script, Section 8F-2): for the 74 shared miRNAs, the AD – Control differences in the two cohorts are essentially **uncorrelated ($r = 0.03$)**, and only 43 of 74 change in the same direction. With ~1,300 training samples this is not simple overfitting — the serum-array AD signature does not exist in the whole-blood RNA-seq data. Most published blood miRNA signatures fail to replicate across platforms and sample types; reporting such a result honestly is good science.

5.6.3 The GSE46579 Validation Dataset

GSE46579 characteristics: - Platform: GPL11154 (Illumina HiSeq 2000 — small RNA-seq); counts from miRDeep2 (miRBase v18) - Sample type: Whole blood - Groups: AD (n=48), Controls (n=22); 65 samples (44 AD, 21 Control) after Week 2 QC - Key difference from GSE120584: RNA-seq vs microarray; whole blood vs serum; much smaller N

The cross-platform challenge: The two datasets name miRNAs using different miRBase versions. GSE46579's counts were assigned with miRBase **v18** (e.g. hsa-miR-98, hsa-let-7c), while the GSE120584 array annotation matches miRBase **v21** (hsa-miR-98-5p, hsa-let-7c-5p). Many miRNA names changed between versions — strand designations (-3p vs -5p), unsuffixed names, and retired entries. A plain name match finds only 69 shared miRNAs.

Before any cross-platform comparison, miRNA names must be harmonized to the same version. Module 5.7 covers this in detail.

MODULE 5.7 — Harmonizing Datasets for Cross-Platform Validation

5.7.1 The miRNA Name Problem

miRBase is the authoritative registry for miRNA names and sequences. It has gone through 22 major versions since 2002. Each version changed names for a subset of miRNAs: - Version 6 → 12: Major restructuring of naming conventions - Version 14 → 16: Strand designation introduced (-3p and -5p suffixes replaced * notation) - Version 17 → 21: Many isomiR and precursor changes - Version 21 → 22: Removal of some miRNAs later identified as non-canonical

Practical consequence: The miRNA named “hsa-miR-21*” in an older array annotation corresponds to “hsa-miR-21-3p” in miRBase v22. If you try to intersect features across your two datasets without harmonization, you will lose a significant fraction of your shared miRNAs — not because they are absent from one platform, but because they are named differently.

The R package `miRBaseConverter` resolves this by converting miRNA names between any two miRBase versions using the miRBase accession number (MIMAT) as a stable identifier.

5.7.2 miRBaseConverter Name Harmonization in R

The full harmonization workflow is implemented in the companion R script (`Week5_Validation.R`). Here is the conceptual walkthrough:

```
library(miRBaseConverter)

# 1. Detect the likely miRBase version of each dataset's name format
version_120584 <- checkMiRNAVersion(rownames(expr_gse120584), verbose = TRUE)
version_46579  <- checkMiRNAVersion(rownames(expr_gse46579),  verbose = TRUE)

#   → "v21" for GSE120584, "v18" for GSE46579

# 2. Look each name up in ITS OWN miRBase version to get the stable
#    MIMAT accession. Looking a v18 name such as "hsa-miR-98" up in v22
#    (where it is "hsa-miR-98-5p") would fail.
#    miRNA_NameToAccession() returns two columns: miRNAName_<version>, Accession
result_120584 <- miRNA_NameToAccession(rownames(expr_gse120584), version = version_120584)
result_46579  <- miRNA_NameToAccession(rownames(expr_gse46579),  version = version_46579)

# 3. Keep mature-miRNA accessions only (MIMAT...); "MI..." accessions are
#    precursors (hairpins), not the mature miRNA that was measured
result_120584$Accession[!grepl("^MIMAT", result_120584$Accession)] <- NA
result_46579$Accession[!grepl("^MIMAT", result_46579$Accession)]  <- NA
# Result: 907 / 925 GSE120584 names and 270 / 273 GSE46579 names map.
# Failures: composite array probes ("hsa-miR-365a-3p, hsa-miR-365b-3p") and
# entries retired before v22 (e.g. hsa-miR-720, later shown to be a tRNA fragment)

# 4. Find the intersection of MIMAT accessions (version-independent)
common_MIMAT <- intersect(
  result_120584$Accession[!is.na(result_120584$Accession)],
  result_46579$Accession[!is.na(result_46579$Accession)]
)
cat("miRNAs in common after harmonization:", length(common_MIMAT), "\n") # 74

# 5. Subset both expression matrices to the intersection
expr_120584_sub <- expr_gse120584[result_120584$Accession %in% common_MIMAT, ]
expr_46579_sub  <- expr_gse46579[ result_46579$Accession %in% common_MIMAT, ]

# 6. Convert MIMAT accessions back to v22 canonical names for readability
v22_names <- miRNA_AccessionToName(common_MIMAT, targetVersion = "v22")
rownames(expr_120584_sub) <- v22_names$TargetName
rownames(expr_46579_sub)  <- v22_names$TargetName
```

5.7.3 Per-Dataset Z-Score Standardization

After finding the intersection, the expression values are on completely different scales: GSE120584 is normalized log2 microarray intensity (≈ -1 to 16); GSE46579 is VST-transformed RNA-seq

counts (≈ 0 to 25). You cannot train on one and test on the other without removing this scale difference.

Z-score standardization per dataset resolves this by converting each dataset independently to have mean = 0 and standard deviation = 1 per miRNA:

```
# =====
# Z-score standardization per dataset
# (Done separately for training and validation datasets)
# =====

# Row-wise z-score: each row (miRNA) has mean=0, SD=1 within its own dataset
z_score_rows <- function(mat) {
  row_means <- rowMeans(mat, na.rm = TRUE)
  row_sds <- apply(mat, 1, sd, na.rm = TRUE)
  keep <- row_sds > 0 # remove zero-variance features
  mat <- mat[keep, ]
  sweep(sweep(mat, 1, row_means[keep], "-"), 1, row_sds[keep], "/")
}

# Z-score EACH DATASET INDEPENDENTLY
# Rationale: do not pool training and validation before z-scoring –
# that would shift the validation data toward the training mean (leakage).
# Each dataset is standardized to have mean=0, SD=1 per miRNA WITHIN that dataset.
# The biological signal (relative differences between AD and Control within each
# dataset) is preserved. The platform-specific absolute scale is removed.

expr_120584_z <- z_score_rows(expr_120584_sub) # training: z-scored
expr_46579_z <- z_score_rows(expr_46579_sub) # validation: z-scored separately

# Verification
cat("Training matrix (z-scored):", nrow(expr_120584_z), "miRNAs x",
    ncol(expr_120584_z), "samples\n")
cat(" Row means (should be ~0): min =",
    round(min(rowMeans(expr_120584_z)), 4), "\n")
cat(" Row SDs (should be ~1): min =",
    round(min(apply(expr_120584_z, 1, sd)), 4), "\n")

cat("Validation matrix (z-scored):", nrow(expr_46579_z), "miRNAs x",
    ncol(expr_46579_z), "samples\n")
```

Why standardize each dataset separately? Consider: a miRNA might have a mean log2 array intensity of 8.7 in GSE120584 but a mean VST expression of 12.4 in GSE46579. These numbers are not comparable — they are on completely different scales due to the different measurement technologies. After per-dataset z-scoring, both datasets will express miR-21-5p as deviations from its own dataset's mean. A sample with high miR-21-5p expression relative to the rest of GSE120584 will have a positive z-score, as will a sample with high miR-21-5p relative to the rest of GSE46579. Relative (within-dataset) differences are preserved; absolute (across-dataset) scale differences are removed.

5.7.4 External Validation: Train on GSE120584, Apply to GSE46579

```
library(randomForest)
library(pROC)

# Prepare training labels (binary: AD vs Control)
# X_train: samples x miRNAs (transposed from expr_120584_z)
```

```

# y_train: factor with levels c("Control", "AD")
# (see Week5_Validation.R Section 6A for data preparation)

set.seed(42)

# Train final model on ALL training data (no CV – for validation prediction)
rf_final <- randomForest(
  x      = X_train,
  y      = y_train,
  ntree  = 500,
  mtry   = floor(sqrt(ncol(X_train))),
  importance = TRUE
)

# Predict on external validation cohort
# X_val: samples x miRNAs (transposed from expr_46579_z, same column order as X_train)
prob_val <- predict(rf_final, newdata = X_val, type = "prob")[, "AD"]

# External validation AUC
roc_val  <- roc(y_val_binary, prob_val, direction = "<", quiet = TRUE)
val_auc  <- as.numeric(auc(roc_val))
ci_val   <- as.numeric(ci.auc(roc_val, method = "bootstrap", boot.n = 2000))

cat(sprintf("\nExternal Validation AUC (GSE46579): %.4f (95%% CI: %.4f–%.4f)\n",
            val_auc, ci_val[1], ci_val[3]))

# ROC curve (ggplot2)
roc_df <- data.frame(
  FPR = 1 - roc_val$specificities,
  TPR = roc_val$sensitivities
)

p_roc_val <- ggplot(roc_df, aes(x = FPR, y = TPR)) +
  geom_line(colour = "#D73027", linewidth = 1.5) +
  geom_abline(intercept = 0, slope = 1, colour = "grey50",
              linetype = "dotted", linewidth = 0.6) +
  annotate("text", x = 0.65, y = 0.20,
          label = sprintf("AUC = %.3f\n(95%% CI: %.3f–%.3f)",
                          val_auc, ci_val[1], ci_val[3]),
          size = 4, colour = "#D73027") +
  labs(
    x      = "False Positive Rate",
    y      = "True Positive Rate",
    title  = "ROC Curve – External Validation on GSE46579\n(Model trained on GSE120584)"
  ) +
  theme_bw(base_size = 12)

ggsave("results/Week5/roc_external_validation.png",
        p_roc_val, width = 6, height = 5.5, dpi = 150)

```

5.7.5 ComBat for Batch Correction Between Studies

For datasets where the sample types are compatible (e.g., both serum), ComBat can be applied to remove cross-study batch effects before combining datasets. This is more aggressive than per-dataset z-scoring and should be used cautiously.

Key considerations: - ComBat-seq requires raw count data (not VST/log-transformed) — so it cannot be used when one dataset is a microarray, as here - The group variable (AD vs Control) must be specified as a biological covariate to protect it from removal - After ComBat, re-run VST transformation and re-check PCA for remaining batch structure - If the two datasets differ in sample type (serum vs whole blood), ComBat will correct technical differences but cannot resolve the genuine biological difference in miRNA composition between sample types

MODULE 5.8 — Reporting ML Results: What Makes a Good Paper

5.8.1 TRIPOD: Transparent Reporting of a Multivariable Prediction Model

TRIPOD (Transparent Reporting of a multivariable prediction model for Individual Prognosis Or Diagnosis) is an international reporting guideline specifically designed for studies that develop or validate prediction models. It was published in the Annals of Internal Medicine (Collins et al., 2015) and has since been adopted by most major clinical journals.

Key TRIPOD items for miRNA ML biomarker studies:

TRIPOD Item	What to Report
4a	Source of data (GEO accession, sample type, collection dates)
4b	Eligibility criteria for participants (diagnosis method, exclusions)
5	Outcome to be predicted (AD diagnosis, conversion from MCI)
7	Sample size justification (power calculation or minimum N for stable AUC)
8	How missing data were handled
10a	Predictors used and how they were selected/transformed
10b	How features were handled (normalization, filtering)
12	Model development details: algorithm, hyperparameters, selection method
13b	Internal validation method: k-fold, LOOCV, nested CV
15	Model performance: discrimination (AUC), calibration, decision curve analysis
16	For validation studies: description of validation data, sources of difference from development data
17	Comparison of development vs validation performance
22	Limitations: potential bias, sample size, generalizability

5.8.2 What Metrics to Report

A complete reporting of a miRNA biomarker ML model should include:

Discrimination: - AUC (with 95% confidence interval) - Sensitivity and specificity at the operating threshold - Positive Predictive Value (PPV) and Negative Predictive Value (NPV) at the operating threshold - Youden index (optimal threshold selection)

Calibration: - Calibration plot (observed vs predicted probability) - Brier score - Hosmer-Lemeshow test (for binary classifiers)

Validation: - Internal CV AUC (from nested CV) - External validation AUC (if independent cohort available) - Bootstrap AUC comparison test p-value comparing training CV and validation AUC

Feature reporting: - Top features (miRNAs) with effect direction (SHAP values, not just importance ranks) - Biological annotation of top features

5.8.3 Confidence Intervals on AUC and DeLong's Test

A single AUC value without a confidence interval is incomplete. The most common method for CI computation is **bootstrap resampling**. For comparing two AUC values (e.g., training CV vs external validation, or Model A vs Model B on the same patients), **DeLong's method** provides an analytic test based on the covariance of correlated ROC curves.

```
library(pROC)

# =====
# Bootstrap confidence interval for AUC (pROC)
# =====

set.seed(42)
roc_obj <- roc(y_val_binary, prob_val, direction = "<", quiet = TRUE)
ci_boot <- ci.auc(roc_obj, method = "bootstrap", boot.n = 2000, conf.level = 0.95)

cat(sprintf("Validation AUC: %.4f (95%% CI: %.4f – %.4f)\n",
            as.numeric(auc(roc_obj)), ci_boot[1], ci_boot[3]))

# Visualize bootstrap distribution (manual bootstrap for illustration)
set.seed(42)
n <- length(y_val_binary)
boot_aucs <- replicate(2000, {
  idx <- sample(n, n, replace = TRUE)
  y_b <- y_val_binary[idx]
  p_b <- prob_val[idx]
  if (length(unique(y_b)) < 2) return(NA_real_)
  as.numeric(pROC::auc(pROC::roc(y_b, p_b, quiet = TRUE)))
})
boot_aucs <- boot_aucs[!is.na(boot_aucs)]

p_boot <- ggplot(data.frame(auc = boot_aucs), aes(x = auc)) +
  geom_histogram(bins = 50, fill = "#4575B4", alpha = 0.7, colour = "white") +
  geom_vline(xintercept = as.numeric(auc(roc_obj)),
            colour = "#D73027", linewidth = 1.5,
            linetype = "solid") +
  geom_vline(xintercept = quantile(boot_aucs, c(0.025, 0.975)),
            colour = "darkorange", linewidth = 1.2, linetype = "dashed") +
  labs(
    x = "Bootstrap AUC",
    y = "Count",
    title = "Bootstrap Distribution of External Validation AUC"
  ) +
  theme_bw(base_size = 11)

ggsave("results/Week5/bootstrap_auc_distribution.png",
       p_boot, width = 7, height = 4, dpi = 150)

# =====
# DeLong / bootstrap AUC comparison test (pROC::roc.test)
```

```
# =====
# For PAIRED comparisons (same patients, two models): method = "delong"
# For INDEPENDENT cohorts (different patients): method = "bootstrap"
# Our two cohorts are independent (different patients) → use bootstrap

roc_train_cv <- roc(cv_preds_rf$true_binary, cv_preds_rf$prob_AD,
                    direction = "<", quiet = TRUE)
roc_val_ext  <- roc(y_val_binary, prob_val_rf,
                    direction = "<", quiet = TRUE)

set.seed(42)
auc_test <- roc.test(
  roc1      = roc_train_cv,
  roc2      = roc_val_ext,
  method    = "bootstrap",
  boot.n    = 2000,
  alternative = "greater",  # H1: training CV AUC > validation AUC
  paired    = FALSE       # independent cohorts
)

cat("\nBootstrap AUC comparison test (training CV vs external validation):\n")
print(auc_test)
cat("\nIf p < 0.05: training AUC is significantly higher than validation AUC.\n")
cat("This is expected for cross-platform validation and does not negate the\n")
cat("scientific value of the model if validation AUC > 0.70.\n")
```

5.8.4 Calibration Plots

A classifier with high AUC but poor calibration is clinically dangerous. Calibration measures whether predicted probabilities are accurate: if the model says $P(AD) = 0.8$ for a group of patients, approximately 80% of those patients should truly have AD.

Calibration is assessed with a **calibration plot** (reliability diagram): group patients by their predicted probability into deciles, then plot mean predicted probability (x-axis) vs observed event rate (y-axis). A perfectly calibrated model would fall on the diagonal.

```
library(ggplot2)
library(dplyr)

# Manual calibration plot using quantile-based binning
calibrate_manual <- function(true_labels, pred_probs, n_bins = 10) {
  breaks <- unique(quantile(pred_probs, probs = seq(0, 1, length.out = n_bins + 1)))
  bin_idx <- cut(pred_probs, breaks = breaks, include.lowest = TRUE, labels = FALSE)
  data.frame(pred = pred_probs, true = true_labels, bin = bin_idx) %>%
    group_by(bin) %>%
    summarise(
      n          = n(),
      mean_pred  = mean(pred),
      observed_rate = mean(true),
      se         = sqrt(observed_rate * (1 - observed_rate) / n),
      .groups    = "drop"
    ) %>%
    filter(!is.na(bin))
}

calib_df <- calibrate_manual(y_val_binary, prob_val_rf, n_bins = 10)
```

```

p_calib <- ggplot(calib_df, aes(x = mean_pred, y = observed_rate)) +
  geom_abline(intercept = 0, slope = 1,
             linetype = "dashed", colour = "grey40", linewidth = 0.8) +
  geom_errorbar(aes(ymin = observed_rate - 1.96 * se,
                  ymax = observed_rate + 1.96 * se),
              width = 0.02, colour = "#4575B4", alpha = 0.8) +
  geom_point(aes(size = n), colour = "#4575B4", alpha = 0.9) +
  geom_smooth(method = "loess", se = TRUE, colour = "#D73027",
            fill = "#D73027", alpha = 0.15, linewidth = 1) +
  scale_size_continuous(name = "n (samples in bin)", range = c(2, 7)) +
  scale_x_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  labs(
    x = "Mean Predicted Probability P(AD)",
    y = "Observed Event Rate (Fraction True AD)",
    title = "Calibration Plot – External Validation (GSE46579)",
    caption = "Dashed line = perfect calibration. Error bars = 95% CI."
  ) +
  theme_bw(base_size = 12)

ggsave("results/Week5/calibration_plot_external.png",
      p_calib, width = 6, height = 6, dpi = 150)

# Brier score (quantitative calibration metric)
brier <- mean((y_val_binary - prob_val_rf)^2)
prev <- mean(y_val_binary)
brier_null <- prev * (1 - prev)
cat(sprintf("Brier Score: %.4f (null model: %.4f | scaled: %.4f)\n",
          brier, brier_null, 1 - brier / brier_null))
cat("(Scaled Brier: 1 = perfect; 0 = null model; negative = worse than null)\n")

```

5.8.5 Decision Curve Analysis

Decision Curve Analysis (DCA) evaluates whether a model provides clinical benefit compared to the baseline strategies of “treat everyone” or “treat no one.” It accounts for the relative cost of false positives vs false negatives (the threshold probability below which you would treat).

While full DCA implementation is beyond this module’s scope, the key principle is: a model with AUC = 0.85 that is poorly calibrated may provide LESS clinical benefit than a simpler model with AUC = 0.78 but excellent calibration. DCA plots net benefit as a function of threshold probability. The dcurves R package (CRAN) provides a straightforward implementation.

5.8.6 The STARD Checklist

STARD (Standards for Reporting of Diagnostic Accuracy) is the companion guideline to TRIPOD specifically for diagnostic test accuracy studies. It includes 30 items organized in four sections: title/abstract, introduction, methods, and results. For miRNA biomarker studies that include validation against a clinical diagnostic standard (CDR, MMSE, amyloid PET), STARD compliance is required by most diagnostics-focused journals.

Key STARD items for miRNA biomarker reporting: - How was the reference standard (AD diagnosis) established? (Clinical criteria, biomarker-confirmed, post-mortem pathology) - Were the index test (miRNA model) and reference standard interpreted without knowledge of each other? (Avoidance of review bias) - Was there a pre-specified threshold or was it data-derived? (If data-derived, report the derivation process) - What is the time interval between sample collection and clinical diagnosis?

WEEK 5 LAB SESSION

Lab 5: Nested Cross-Validation, SHAP, and External Validation in R (180 min)

Objective: Run the complete Week 5 pipeline in R — nested CV, SHAP interpretation, external validation, and calibration — without leaving RStudio.

Part 1 — Data harmonization and z-scoring (30 min)

Open Week5_Validation.R and run Sections 1-5: 1. Load both datasets from data/processed/ 2. Run miRBaseConverter harmonization. Note: how many miRNA names changed? How many features were lost at the intersection step? 3. Check that key AD-associated miRNAs (miR-21-5p, miR-29b-3p, miR-155-5p, miR-9-5p) are present in the intersection 4. Apply per-dataset z-score standardization 5. Verify standardization: row means ~0, row SDs ~1

Part 2 — ML classifiers and nested CV (60 min)

Run Section 6: 1. Subset to AD vs Control (binary classification) 2. Set up trainControl with repeatedcv (5-fold $\times$ 3 repeats), twoClassSummary 3. Define the RF hyperparameter grid (mtry values) 4. Run caret::train() for Random Forest — note the best mtry and CV AUC 5. Run caret::train() for LASSO (glmnet, alpha=1) — note the best lambda 6. Extract cross-validated predictions using model\$pred

7. Extend to three-class (AD/MCI/Control) using multiClassSummary
8. Print the confusion matrix with confusionMatrix(..., mode = "everything")

Part 3 — SHAP feature importance (30 min)

Run Section 7: 1. Refit Random Forest on full training data (best mtry from nested CV) 2. Define pfun_rf for fastshap 3. Call fastshap::explain() with pred_wrapper = pfun_rf, nsim = 50 and a 200-sample subset as newdata 4. Plot the beeswarm using ggplot2

5. For the top miRNA by mean |SHAP|: note the direction of effect (does high expression $\rightarrow$ AD or $\rightarrow$ Control?)
6. Look up this miRNA in miRBase (<https://www.mirbase.org>) and record its target gene families

Part 4 — External validation and calibration (30 min)

Run Sections 8-9: 1. Standardize GSE46579 group labels; subset to binary AD vs Control 2. Apply predict() to the validation matrix 3. Compute external validation AUC with pROC::auc() 4. Compare training CV AUC vs validation AUC — is the gap < 0.10 ? 5. Run the bootstrap AUC comparison test 6. Plot calibration curve; compute Brier score

Part 5 — Summary table and outputs (30 min)

Run Sections 10-12: 1. Generate the model performance summary table 2. Confirm results/Week5/shap_feature_importance.csv was saved (required by Week 6) 3. Confirm data/processed/harmonized_expr.rds was saved (required by Week 6)

Deliverable: A summary table in the format:

Model	Training CV AUC (95% CI)	Ext. Validation AUC (95% CI)	AUC Gap	Brier Score
Ran- dom Forest	? (? - ?)	? (? - ?)	?	?
LASSO	? (? - ?)	? (? - ?)	?	?

Reflection questions: - Was the nested CV AUC higher or lower than what you might have expected from a flat CV? Is this the expected direction? - Which model has the smallest AUC gap between training and external validation? Is this the same model with the highest training CV AUC? - What fraction of miRNA features were retained in the intersection? Which biologically

important AD miRNAs were lost (if any)? - The top miRNA by SHAP importance: does its direction of effect (high expression → AD or Control?) match the published literature for that miRNA? - If you were preparing this analysis for a journal submission, what would you report in the limitations section about the cross-platform validation?

WEEK 5 ASSIGNMENTS

Reading Assignment

1. **Collins GS et al. (2015)** — TRIPOD Statement: A Set of Recommendations for Reporting of Studies Developing, Validating, or Updating a Multivariable Clinical Prediction Model. *Ann Intern Med* 162(1):55-63. DOI: [10.7326/M14-0698](https://doi.org/10.7326/M14-0698) Focus on: TRIPOD checklist items 10-17 (model development and validation sections)
2. **Bossuyt PM et al. (2015)** — STARD 2015: An Updated List of Essential Items for Reporting Diagnostic Accuracy Studies. *BMJ* 351:h5527. DOI: [10.1136/bmj.h5527](https://doi.org/10.1136/bmj.h5527) Focus on: items related to reference standard and index test definition
3. **Xu Z et al. (2022)** — Machine learning and complex biological data. *Genome Biol* 23:197. DOI: [10.1186/s13059-022-02754-3](https://doi.org/10.1186/s13059-022-02754-3) Focus on: the discussion of overfitting, evaluation strategies, and reproducibility challenges in omics ML

Reflection Questions

1. You run a nested CV and get mean outer AUC = 0.82. Your collaborator runs a flat CV (hyperparameters tuned on the same data) and reports AUC = 0.91. How would you explain the discrepancy to a clinician who is excited about the 0.91 number? What specific leakage form does flat CV introduce?
2. Your model's SHAP analysis reveals that the top predictive miRNA is hsa-miR-486-5p, which is known to be highly abundant in red blood cells. Your training data comes from serum (depleted of RBCs), but the model assigns it high importance. What does this suggest about the data, and what should you investigate?
3. Your external validation AUC drops from 0.85 to 0.62 (a 0.23 gap). You examine the feature list and find that 40% of the top-30 model features are not present in the validation platform. What are two strategies you could employ to improve generalizability, and what are the biological trade-offs of each?
4. A colleague argues that calibration is irrelevant for a screening test — “all that matters is AUC.” Construct a biological scenario where a high-AUC, poorly-calibrated model would lead to patient harm in an MCI screening context.

Practical Exercise

Using the summary results table from the Lab session, write a “Results” paragraph (4-6 sentences) as you would in a journal paper. Include: the training CV AUC with 95% CI, the external validation AUC with 95% CI, the bootstrap AUC comparison result, the top-3 miRNAs by SHAP value with their effect directions, and one sentence about calibration. Follow TRIPOD item 15 reporting standards.

WEEK 5 GLOSSARY

Term	Definition
Overfitting	When a model captures the noise patterns specific to the training data and performs substantially worse on new data
Bias-variance tradeoff	The inverse relationship between a model's error from overly rigid assumptions (bias) and its error from sensitivity to training data specifics (variance)
Data leakage	Any flow of information from the test or validation set into model training or feature selection, causing inflated performance estimates
Nested cross-validation	A CV design with two loops: the outer loop estimates unbiased performance; the inner loop selects hyperparameters using only the outer training fold
Stratified k-fold CV	A k-fold CV variant that ensures each fold contains the same class proportions as the full dataset; required for imbalanced classification problems
LOOCV	Leave-One-Out Cross-Validation; $k = N$; each sample serves as a test set exactly once; preferred for very small datasets
LASSO	Least Absolute Shrinkage and Selection Operator; L1-regularized regression that drives many coefficients to exactly zero, producing sparse, interpretable models
glmnet	R package implementing LASSO, ridge, and elastic-net regularized regression; $\alpha=1$ for LASSO, $\alpha=0$ for ridge
Lambda (λ)	Regularization strength in LASSO/ridge; larger λ = more shrinkage = fewer selected features; optimized by cross-validation
Random Forest	Ensemble of decision trees trained on bootstrap samples with random feature subsets; <code>mtry</code> controls the number of features sampled at each split
mtry	The number of features randomly sampled at each split in a Random Forest; typically set to $\sqrt{p}$ for classification
SHAP values	SHapley Additive exPlanations; assigns each feature a contribution to each individual prediction, grounded in cooperative game theory
fastshap	R package for computing approximate SHAP values for any black-box model using Monte Carlo permutation
Beeswarm plot	A SHAP summary visualization showing each sample-feature combination as a dot, colored by feature value and positioned by SHAP value
Macro AUC	Average AUC across all classes with equal weighting; used in multiclass classification; sensitive to performance on small classes
Weighted AUC	Average AUC across classes weighted by class sample size; less affected by small-class performance
External validation	Evaluation of a trained model on a completely independent dataset, ideally from a different cohort, site, or platform
TRIPOD	Transparent Reporting of a multivariable prediction model for Individual Prognosis Or Diagnosis; international reporting guideline for biomarker ML studies
STARD	Standards for Reporting of Diagnostic Accuracy; reporting guideline for studies validating diagnostic tests against a clinical reference standard
DeLong's method	A statistical test comparing two AUC values that accounts for their correlation when computed on the same patients; for independent cohorts, bootstrap test is used instead

Term	Definition
Calibration plot	A reliability diagram plotting mean predicted probability against observed event rate; a perfectly calibrated model falls on the diagonal
Brier score	Mean squared error of predicted probabilities; 0 = perfect; 0.25 = uninformative (equivalent to always predicting the prevalence)
miRBaseConverter	R package for converting miRNA names between different versions of the miRBase registry using stable MIMAT accession numbers
Per-dataset z-scoring	Standardizing each dataset's expression values independently to mean=0, SD=1 per feature before cross-platform comparison

KEY REFERENCES (Week 5)

1. Collins GS, Reitsma JB, Altman DG, Moons KGM (2015). Transparent Reporting of a Multivariable Prediction Model for Individual Prognosis or Diagnosis (TRIPOD): The TRIPOD Statement. *Ann Intern Med* 162(1):55-63. DOI: [10.7326/M14-0698](https://doi.org/10.7326/M14-0698) — Primary reference for prediction model reporting
2. Bossuyt PM et al. (2015). STARD 2015: An Updated List of Essential Items for Reporting Diagnostic Accuracy Studies. *BMJ* 351:h5527. DOI: [10.1136/bmj.h5527](https://doi.org/10.1136/bmj.h5527) — Reporting standard for diagnostic accuracy studies
3. DeLong ER, DeLong DM, Clarke-Pearson DL (1988). Comparing the Areas under Two or More Correlated Receiver Operating Characteristic Curves: A Nonparametric Approach. *Biometrics* 44(3):837-845. DOI: [10.2307/2531595](https://doi.org/10.2307/2531595) — Statistical method for AUC comparison
4. Ludwig N et al. (2019). Distribution of miRNA expression across human tissues. *Nucleic Acids Res* 47(1):e3. DOI: [10.1093/nar/gky1151](https://doi.org/10.1093/nar/gky1151) — Context for interpreting tissue-specific and blood-based miRNA profiles
5. Xu Z, Marchionni L, Wang JT, et al. (2022). Machine learning and complex biological data. *Genome Biol* 23:197. DOI: [10.1186/s13059-022-02754-3](https://doi.org/10.1186/s13059-022-02754-3) — Review of overfitting and reproducibility in omics ML
6. Varoquaux G (2018). Cross-validation failure: Small sample sizes lead to large error bars. *Neuroimage* 180(Pt A):68-77. DOI: [10.1016/j.neuroimage.2017.06.061](https://doi.org/10.1016/j.neuroimage.2017.06.061) — Quantification of CV variance in small-N studies
7. Lundberg SM, Lee SI (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems* 30:4765-4774. arXiv: [1705.07874](https://arxiv.org/abs/1705.07874) — Original SHAP paper
8. Greenwell B (2023). fastshap: Fast Approximate Shapley Values. R package. CRAN — SHAP implementation used in this course
9. Kuhn M (2008). Building Predictive Models in R Using the caret Package. *J Stat Softw* 28(5):1-26. DOI: [10.18637/jss.v028.i05](https://doi.org/10.18637/jss.v028.i05) — caret package reference
10. Friedman J, Hastie T, Tibshirani R (2010). Regularization Paths for Generalized Linear Models via Coordinate Descent. *J Stat Softw* 33(1):1-22. DOI: [10.18637/jss.v033.i01](https://doi.org/10.18637/jss.v033.i01) — glmnet LASSO implementation
11. Hebert SS et al. (2008). Loss of microRNA cluster miR-29a/b-1 in sporadic Alzheimer's disease correlates with increased BACE1/beta-secretase expression. *Proc Natl Acad Sci USA* 105(17):6415-6420. DOI: [10.1073/pnas.0710263105](https://doi.org/10.1073/pnas.0710263105) — Biological validation for miR-29 SHAP interpretation

12. Cogswell JP et al. (2008). Identification of miRNA Changes in Alzheimer's Disease Brain and CSF Yields Putative Biomarkers and Insights into Disease Pathways. *J Alzheimers Dis* 14(1):27-41. DOI: [10.3233/JAD-2008-14103](https://doi.org/10.3233/JAD-2008-14103) — *Context for AD miRNA biology and biomarker interpretation*
 13. Robin X et al. (2011). pROC: an open-source package for R and S+ to analyze and compare ROC curves. *BMC Bioinformatics* 12:77. DOI: [10.1186/1471-2105-12-77](https://doi.org/10.1186/1471-2105-12-77) — *DeLong test and ROC analysis in R*
 14. Johnson WE, Li C, Rabinovic A (2007). Adjusting batch effects in microarray expression data using empirical Bayes methods. *Biostatistics* 8(1):118-127. DOI: [10.1093/biostatistics/kxj037](https://doi.org/10.1093/biostatistics/kxj037) — *ComBat methodology for cross-study harmonization*
-

Next Week Preview (Week 6)

Week 6: From Discovery to Clinical Translation

Next week, we bridge the gap between a validated computational model and clinical deployment. We will cover:

- **Biomarker qualification frameworks:** FDA Biomarker Qualification Roadmap; EMA guidance on biomarker-driven trials; the IVD (In Vitro Diagnostic) regulatory pathway
- **From miRNA panel to clinical assay:** How a serum miRNA panel discovered by ML is translated to an RT-qPCR multiplex assay; what changes between discovery and clinical formats
- **Prospective validation design:** How to design a prospective validation study (sample size calculation, pre-specified thresholds, blinded evaluation)
- **Biological follow-up experiments:** How to use your SHAP-identified top miRNAs as entry points for functional validation (luciferase reporter assays, miRNA mimic/inhibitor transfection, pathway enrichment analysis)
- **Integrative biomarkers:** Combining miRNA with clinical variables (MMSE, age, APOE genotype) in a multi-modal model; multi-omics integration conceptual overview
- **Week 6 reads from Week 5 outputs:** `results/Week5/shap_feature_importance.csv` and `data/processed/harmonized_expr.rds`